

Recommendation Algorithms

Project Reference — movie-recommender (backend) + cine-craft-box integration

Use this document as the canonical reference for how TMDB, your content-based scorer, and the hybrid blender work together, including personalization, API surface, and frontend consumption.

Date: August 25, 2026 | **Stack:** Python / FastAPI + TMDB + React (cine-craft-box) | **Principle:** TMDB supplies what exists; you decide order and relevance.

1. High-level Design

Your system is a **hybrid recommender** composed of three layers:

- **Candidate generation** — TMDB (search, recommendations, similar, discover, top Rated)
- **Scoring / re-ranking** — your content-based logic (genre, year, rating)
- **Blending** — weighted mix of TMDB popularity signals + content similarity
- **Personalization** — user favorites (IP-keyed) seed the candidate pool and scoring

Principle: TMDB supplies what exists; you decide order and relevance.

2. Content-based Layer (movie_recommender.py)

Core idea: score how similar movie A is to movie B using metadata only (no collaborative filtering / no user-user matrix).

2.1 Jaccard Similarity (genres)

$$J(A, B) = |G_A \cap G_B| / |G_A \cup G_B|$$

G_A, G_B = sets of genre labels

Range: 0 (no overlap) $\rightarrow$ 1 (identical genre sets)

Good for: "same kind of movie" in pure genre terms

2.2 Feature Vector + Cosine Similarity

Each movie becomes a numeric vector, typically:

- Multi-hot (or similar) encoding of genres
- Normalized year
- Normalized rating

$$\cos(a, b) = (a \cdot b) / (||a|| * ||b||)$$

Captures joint similarity across genres + year + rating in one score. Metric name in API: **cosine**

2.3 Weighted Score (default metric)

Default blend used in most paths:

```
score = 0.5 * genre_overlap + 0.2 * year_proximity + 0.3 * rating_closeness
```

Component	Weight	Role
Genre overlap (Jaccard-style)	0.5	Strongest signal — 'same type of film'
Year proximity	0.2	Prefer similar era
Rating closeness	0.3	Prefer similar quality band

Interpretation: High score → same genres, nearby year, similar rating. Low score → different genres / era / quality.
Metric name in API: **weighted (default)**.

2.4 Functions

Function	Role
compute_jaccard_similarity	Genre-set overlap
build_feature_vector	Vector for cosine
compute_cosine_similarity	Vector similarity
compute_weighted_score	Default 0.5 / 0.2 / 0.3 blend
get_content_recommendations	Rank candidates vs one target movie
get_personalized_content_recommendations	Rank candidates vs a list of favorites (best match per candidate)

Personalized content rule of thumb: For each candidate, score against each favorite; keep the best (or aggregated) similarity so one strong match is enough.

3. Hybrid Layer (api.py)

3.1 Pipeline

```
User context (favorites / tmdb_id / filters)
↓
TMDB candidate pool
- /movie/{id}/recommendations
- /movie/{id}/similar
- /discover/movie (diversity / filters)
↓
convert_tmdb_to_content_format()
↓
Content scoring (weighted / jaccard / cosine)
↓
TMDB score (popularity + vote volume, log-normalized)
↓
hybrid_score = tmdb_weight * tmdb_score + content_weight * content_score
```



Sort by `hybrid_score` → return top N

3.2 TMDB Score (normalized)

Roughly:

- Popularity → log-scaled into $\sim[0, 1]$
- Vote count → log-scaled into $\sim[0, 1]$
- Often averaged into `tmdb_score`

Log scaling reduces domination by extreme blockbusters.

3.3 Hybrid Formula

$$\text{hybrid_score} = w_t * \text{tmdb_score} + w_c * \text{content_score}$$

Weight	Default	Effect
tmdb_weight (<code>w_t</code>)	0.4	Popularity / social proof
content_weight (<code>w_c</code>)	0.6	Your similarity logic

Tunable per request, e.g.:

`/api/recommendations/hybrid?tmdb_id=550&tmdb_weight=0.3&content_weight=0.7`

Higher `content_weight` → more 'like my favorites / this movie'

Higher `tmdb_weight` → more broadly popular

3.4 Helper: `convert_tmdb_to_content_format`

Maps TMDB fields → structure expected by content functions: **name**, **year**, **genre / all_genres**, **rating**, **poster_url**, **id** plus **tmdb_popularity**, **tmdb_rank** for blending. Includes `ensure_search_fields()` for consistent local-style fields.

4. Personalization (favorites)

4.1 How Identity Works Today

Favorites for ranking are stored per client IP in disk cache: `fav_keys_{ip}`

Global `favorites.json` is secondary (local-dataset titles)

Frontend also keeps `localStorage` for UI hearts

4.2 Add / Remove Behavior

Action	Behavior
POST <code>/api/favorites</code>	Always <code>add_user_favorite_key(ip, name, year)</code> . Tries local-file <code>add_favorite</code> (may fail if not in dataset)
DELETE <code>/api/favorites</code>	Always clears IP key. Tries global file remove. Still returns 200 if only IP key existed.

4.3 Building Seeds: build_user_favorite_movies(ip)

For each (name, year) in IP keys:

- If in _movies_map → use full local record
- Else → stub { name, year, genre, rating≈7, ... } so scoring still runs

TMDB resolution happens when fetching candidates (tmdb_search_movie → recommendations/similar)

4.4 Personalized Hybrid Path (GET /api/recommendations)

- Load fav_keys for IP
- If empty → fallback: TMDB top_rated ("Popular on TMDB (no favorites yet)")
- If non-empty: For up to ~5 favorites → TMDB search → recommendations + similar. Deduplicate candidates. Content-score vs favorite set. Filter out already-favorited titles (optional). Return ranked list with based_on.source ≈ hybrid_tmdb_content

4.5 Caching

Personalized response cache key includes: user_ip, fav_keys (sorted), limit, include_viewed. So adding/removing a favorite changes the key → new result, no manual cache bust required. TTL ~30 minutes is fine.

Note: Free Render disk cache can reset on sleep/redeploy → users may need to heart again.

5. API Surface (recommendation-related)

Endpoint	Algorithm	Primary Use
GET /api/recommendations	Personalized hybrid from favorites	Home 'Recommended for You'
GET /api/recommendations/hybrid	Hybrid by tmdb_id and/or genre/year/rating	Detail 'Similar', discovery
GET /api/recommend/content/by-title	Pure content (local + TMDB title match)	Content-only experiments
GET /api/recommend/content/personalized	Content vs local favorites	Local-dataset path
GET /api/recommendations/discovery	High-rated, outside usual genres	'Hidden gems'
GET /api/movies/{id}/recommendations	Pure TMDB	Legacy / fallback
GET /api/recommend/by-title	Pure TMDB by title	Legacy

Response shape (hybrid / personalized):

```
{
  "recommendations": [
    {
      "movie": { "id", "name", "year", "genre", "rating", "poster_url", ... },
      "similarity_score": 0.82,
      "match_reason": "Hybrid: TMDB score 0.45 + Content score 0.90"
    }
  ],
  "based_on": { "source": "...", "favorites_count": 2, ... },
  "total_available": 12
}
```

Frontend should parse: item.movie ?? item (nested movie object).

6. Frontend Wiring (cine-craft-box)

UI	Endpoint
Home 'Recommended for You'	GET /api/recommendations
Detail similar (after upgrade)	GET /api/recommendations/hybrid?tmdb_id=...
Hearts	POST / DELETE /api/favorites + localStorage
Refetch trigger	favorites.length → favoritesVersion

Parsing must prefer nested movie fields so hybrid payloads don't wipe name and hide the section.

7. Algorithms You Are Not Using (yet)

Useful to know for future upgrades:

Technique	What it is	Why you might add it
Collaborative filtering	'Users like you also liked...'	Needs many users + interaction history
Matrix factorization / embedding	embed users from ratings	Strong at scale; heavier infra
Text / NLP similarity	Embed overviews/plots	Better 'same vibe' beyond genres
Session-based	Last N clicks	Better cold-start without permanent favorites
Learning-to-rank	Train on clicks/watches	Optimizes real engagement

Your stack today is content-based + TMDB popularity hybrid – solid for a product without a large rating graph.

8. Strengths and Limits

Strengths

- Works with sparse users (favorites only)
- Explainable (match_reason, scores)
- You own ranking (not a pure TMDB proxy)
- Tunable weights
- Handles TMDB titles outside local CSV

Limits

- IP identity ≠ real accounts (shared Wi-Fi, Render restarts)
- Genre/year/rating miss plot/tone/cast nuance
- Candidate quality still depends on TMDB
- Free-tier cache/sleep can drop IP favorites
- No true 'people like you' collaborative signal

9. Operational Checklist

Healthy personalized request

- POST /api/favorites → 200
- GET /api/recommendations → 200
- Logs show TMDB search / recommendations / similar, not only top Rated
- Payload: favorites_count > 0, hybrid-style match_reason

Fallback path

- No IP favorites → top Rated + reason 'Popular on TMDB (no favorites yet)'

10. One-Paragraph Summary

You implemented a hybrid movie recommender: TMDB builds a candidate set; a content-based scorer (mainly weighted genre 50% + year 20% + rating 30%, with Jaccard and cosine variants) measures similarity to a target or to the user's favorites; a blended hybrid score (default 40% TMDB popularity, 60% content) re-ranks results. Personalization keys favorites by client IP, accepts titles outside the local dataset, and caches by favorite set so list changes track hearts without manual cache invalidation.

End of reference. Keep this PDF in repo docs/ or Notion for onboarding and future upgrades.